

Seminário SQLMap

 por Matheus Barros

SQL Injection

OWASP

A Injeção de SQL é categorizada como A03:2021 no OWASP Top 10 (Injection), ressaltando sua gravidade e prevalência.

Exemplo

```
SELECT * FROM users WHERE id = '1 OR 1=1'
```

A aplicação confia cegamente nos dados fornecidos pelo usuário e os insere diretamente em uma query SQL.

Vulnerabilidade

É uma falha que permite a manipulação de consultas enviadas a bancos de dados, alterando seu comportamento original.

Consequências

As ramificações incluem acesso não autorizado a dados sensíveis, escalonamento de privilégios e até controle total do sistema subjacente.

Exemplo de Código Vulnerável

```
// Rota para buscar um produto por ID
app.get('/products/:id', async (req, res) => {
  const productId = req.params.id;
  // VULNERÁVEL: O ID é inserido diretamente na string!
  const query = `SELECT * FROM products WHERE id = ${productId}`;
  const result = await db.query(query);
  res.json(result.rows);
});
```

O que é SQLMap?

CLI

Ferramenta Python

Aplicação de linha de comando

6

Técnicas de Injeção

Incluindo Union-based, Error-based e Blind SQLi.

30+

DBMS Suportados

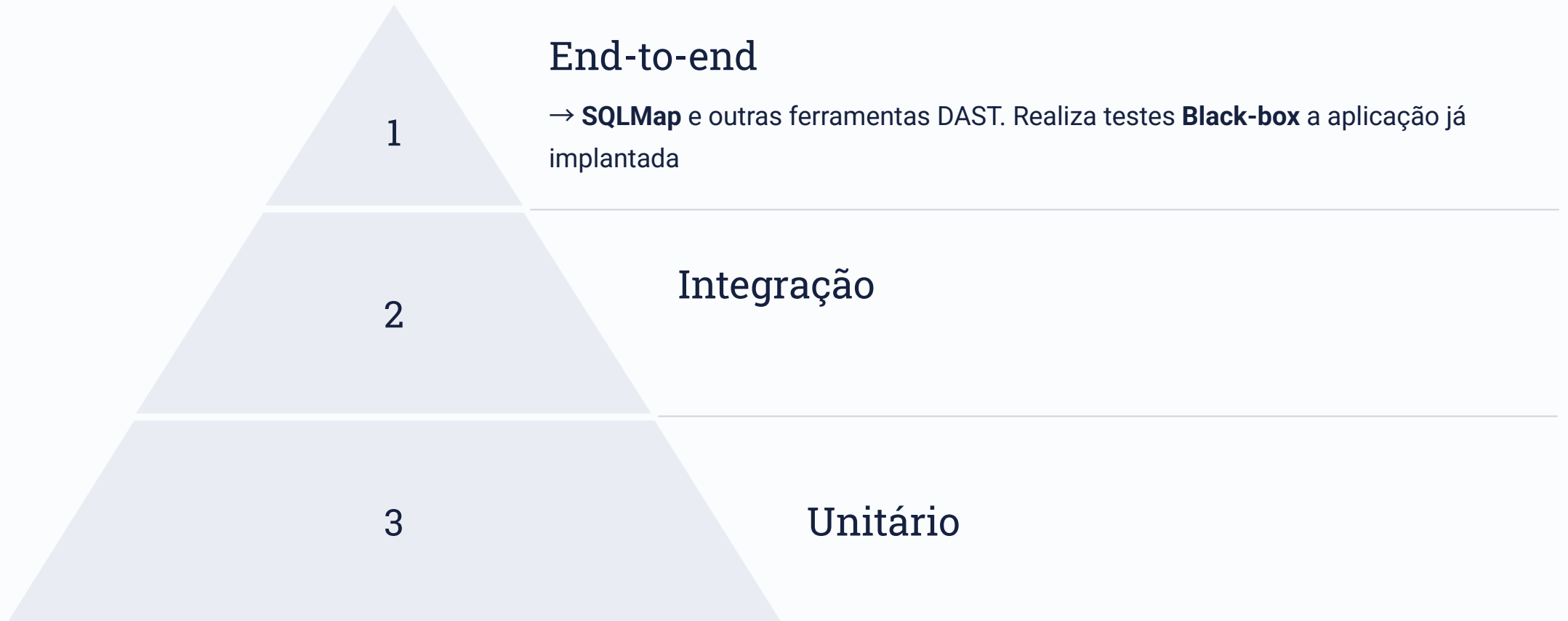
Como MySQL, PostgreSQL e SQL Server.

1000+

Requisições/Segundo

Projetado para automação e eficiência.

Pirâmide de Testes



Funcionalidades

1 Enumeração Abrangente

Permite a enumeração detalhada de bancos de dados, tabelas, colunas e usuários, revelando a estrutura interna do DBMS.

2 Descarregamento de Dados

Facilita o descarregamento (dump) completo de dados de tabelas específicas, incluindo informações confidenciais.

3 Exploração de Senhas

Capaz de detectar e explorar hashes de senhas, com a opção de quebrar essas senhas usando ataques de dicionário.

4 Manipulação de Arquivos

Suporta a leitura e escrita de arquivos no servidor, possibilitando, por exemplo, o upload de web shells para controle remoto.

5 Acesso ao Sistema Operacional

Em sistemas vulneráveis, permite o acesso direto ao sistema operacional subjacente via OS shell ou integração com Meterpreter.

Como o SQLMap Funciona

1

Análise de Parâmetros

Inicia com a análise da URL ou requisição HTTP em busca de parâmetros potencialmente vulneráveis à injeção.

2

Testes com Payloads

Aplica uma série de payloads específicos para cada tipo de injeção, observando as respostas do servidor para identificar falhas.

3

Identificação do DBMS

Automaticamente identifica o sistema de gerenciamento de banco de dados, sua versão e o usuário atualmente em uso.

4

Extração Iterativa

Procede com a extração de informações de forma passo a passo, construindo o ataque com base nas descobertas preliminares.

5

Contorno de WAFs

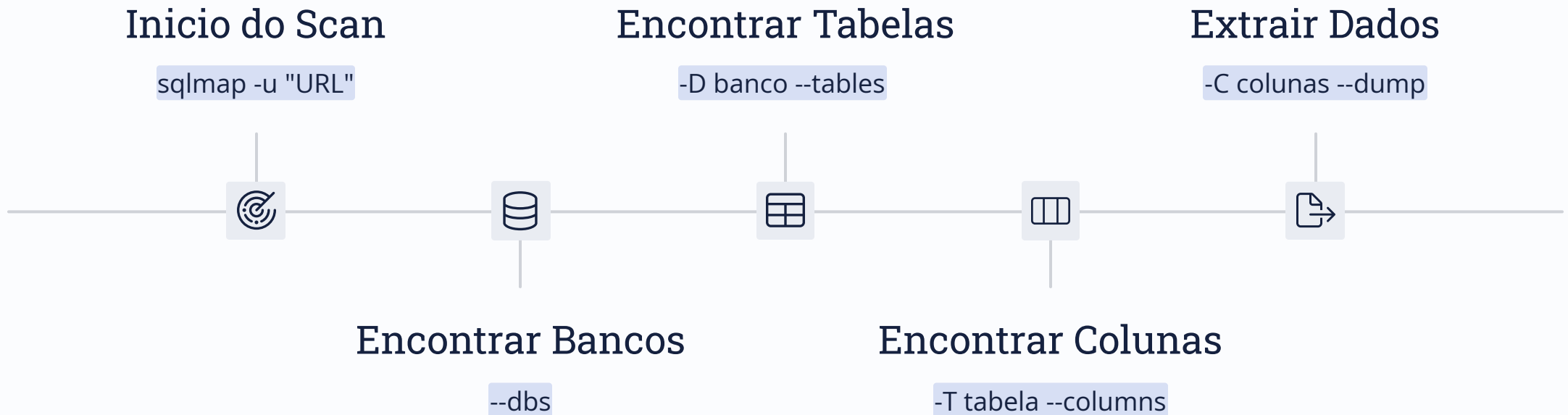
Pode empregar técnicas de ofuscação e evasão, utilizando o parâmetro `--tamper` para tentar contornar Web Application Firewalls (WAFs).

Comandos Essenciais

Exemplos de comandos práticos para operar o SQLMap em cenários comuns de teste de penetração:

<code>sqlmap -u "URL"</code>	Detecção de vulnerabilidades em uma URL específica.
<code>--dbs</code>	Lista todos os bancos de dados disponíveis.
<code>-D "DB" --tables</code>	Lista tabelas em um banco de dados selecionado.
<code>-D "DB" -T "users" --dump</code>	Descarrega dados da tabela 'users' no DB especificado.
<code>sqlmap -r "arquivo.req"</code>	Testa uma requisição complexa (POST, com cookies) salva a partir do Burp Suite.
<code>--os-shell</code>	Obtém um shell de comando no sistema operacional alvo (se vulnerável).

Fluxo de Ataque



Integrações

Ferramenta	Integração
GitHub Action	Possível utilizar <i>action-sqlmap</i>
JUnit	Leitura de resultados JSON
BurpSuite	Exporta requests que podem ser usados como parâmetro

Vantagens x Desvantagens

✓ **Cobertura:** técnicas, suporte a muitos bancos, *tamper scripts* prontos.

✓ **Curva inicial:** curta (-u "URL" --dump) já funciona. Existem muitas Cheat-sheets.

✓ **Maturidade:** ativo desde 2006, centenas de *commits*/ano.

✗ **Agressividade:** pode gerar ruído em logs; payloads podem derrubar app frágil.

✗ **CI/CD:** falta guia oficial de integração; depende da comunidade.

✗ **CLI:** não possui nativamente interface amigável para uso.

Legalidade e Limitações

Permissão Explícita

O uso do SQLMap é estritamente limitado a sistemas para os quais você possui permissão expressa e por escrito do proprietário. Ações não autorizadas são ilegais e passíveis de penalidades.

Implicações Legais no Brasil

No Brasil, a Lei nº 12.737/2012 (Lei Carolina Dieckmann) tipifica crimes informáticos, prevendo graves consequências jurídicas para invasões e acessos indevidos.

Nem todas as Injeções

O SQLMap não é uma solução mágica; nem todas as injeções SQL são automaticamente exploráveis, especialmente em cenários complexos ou com WAFs robustos.

Conhecimento Essencial

Apesar de sua automação, o uso efetivo do SQLMap requer conhecimento básico sólido de redes, segurança da informação e princípios de banco de dados.

Casos Reais

Caso	Vulnerabilidade	Impacto
Heartland Payment Systems (2008)	Formulário web antigo permitia SQL	130M de cartões de crédito expostos
Sony Pictures Entertainment (2011)	SQLi para exfiltrar bases sem hash de senha	1M contas vazadas
TalkTalk (2015)	Sistema de Banco de Dados fora de suporte	157k clientes afetados. Prejuízo de £60M
Illinois State Board of Elections (2016)	SQLi na página de status do registro de eleitores	200k registros de eleitores expostos

Ferramentas Similares

jSQL Injection

GUI Java, automatiza SQLi em
Windows/Mac/Linux

Sqlninja

Foca em SQLi no SQL Server, com *priv-escalation* em Windows.

NoSQLMap

Versão voltada a MongoDB

Quando Usar x Evitar

✓ **Teste de segurança:** DAST em APIs e sites antes de ir para produção

✓ **Ethical Hacking:** durante *bug bounty*

✓ **Aprendizado:** laboratórios de vulnerabilidades para estudo prático

✗ **Aplicações críticas em produção:** sem ambiente de staging, cargas de teste podem causar DoS

✗ **Unethical Hacking:** uso para fins criminosos

✗ **Exploração não autorizada:** análise de aplicações sem permissão do proprietário